

# A LLM-driven and motif-informed linearizing graph transformer for Web API recommendation

Xin Zheng<sup>a,b</sup>, Guiling Wang<sup>a,b,\*</sup>, Guiyue Xu<sup>a,b</sup>, Jianye Yang<sup>a,b</sup>, Boyang Han<sup>d,e</sup>, Jian Yu<sup>c</sup>

<sup>a</sup> Beijing Key Laboratory on Integration and Analysis of Large-scale Stream Data, Beijing, China

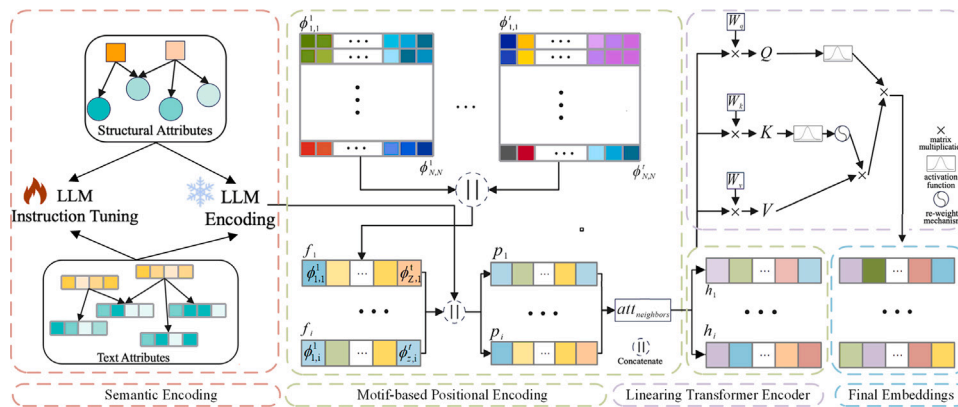
<sup>b</sup> School of Information Science and Technology, North China University of Technology, Beijing, China

<sup>c</sup> Department of Computer Science, Auckland University of Technology, Auckland, New Zealand

<sup>d</sup> Southwest Jiaotong University, Chengdu, China

<sup>e</sup> JD Intelligent Cities Research, China

## GRAPHICAL ABSTRACT



## ARTICLE INFO

### Keywords:

Graph transformer  
Web API recommendation  
Motif  
Higher-order connectivity  
Semantic information

## ABSTRACT

The rapid growth in the number of Web APIs makes it difficult for developers to find the appropriate ones. To tackle this issue, researchers have created various powerful automatic approaches for recommendations. Recently, a range of graph neural networks, drawing inspiration from Transformers, have incorporated global attention to enhance recommendations. However, these approaches still have limitations in terms of processing information between nodes and utilizing other valid information, which reduces their effectiveness in large-scale Web API recommendations. To tackle these problems, this paper introduces a novel positional and semantic encoding method and motif-based linearizing graph Transformer for automatic Web API recommendation. We integrate the semantic information of the nodes into the positional information by using a fine-tuned large language model and graph attention mechanism. Furthermore, we leverage motif information to alter the computational sequence of the Vanilla Transformer, achieving linear time complexity. Experimental results on the two real-world datasets demonstrate the suitability of our model for Web API recommendation, surpassing existing state-of-the-art methods. In summary, our proposed technique exhibits promising results for Web API recommendation and underscores the potential of global attention in this field.

\* Corresponding author at: School of Information Science and Technology, North China University of Technology, Beijing, China.

E-mail addresses: [zhengxin@mail.ncut.edu.cn](mailto:zhengxin@mail.ncut.edu.cn) (X. Zheng), [wangguiling@ncut.edu.cn](mailto:wangguiling@ncut.edu.cn) (G. Wang), [xuguoyue@mail.ncut.edu.cn](mailto:xuguoyue@mail.ncut.edu.cn) (G. Xu), [yangjie@mail.ncut.edu.cn](mailto:yangjie@mail.ncut.edu.cn) (J. Yang), [Hanboyang@my.swjtu.edu.cn](mailto:Hanboyang@my.swjtu.edu.cn) (B. Han), [jian.yu@aut.ac.nz](mailto:jian.yu@aut.ac.nz) (J. Yu).

<https://doi.org/10.1016/j.asoc.2024.112547>

Received 14 July 2024; Received in revised form 18 October 2024; Accepted 15 November 2024

Available online 2 December 2024

1568-4946/© 2024 Elsevier B.V. All rights reserved, including those for text and data mining, AI training, and similar technologies.

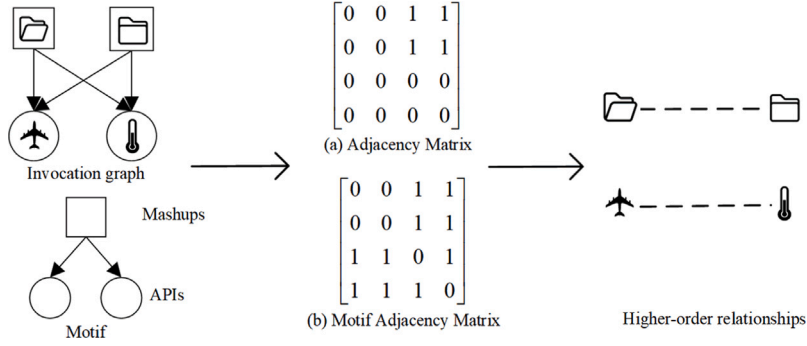


Fig. 1. An example illustrates that for a given graph and motif, the motif adjacency matrix reveals potential relationships between nodes, thereby uncovering previously undiscovered higher-order relationships.

## 1. Introduction

With the advancement of information technology, the diversity and quantity of Web APIs have expanded at an exponential rate. This explosion in APIs poses a substantial challenge for developers who need to integrate them into their mashups, which are web-integrated services that combine multiple Web APIs for specific purposes. As a result, Web API recommendation holds substantial utility across a multitude of application scenarios and projects. Previous studies have seen widespread adoption of Collaborative Filtering (CF) approaches, content-based methods, and hybrid systems [1,2] for recommending Web APIs to developers.

Recently, graph neural networks (GNNs) have surged in popularity within the field of recommender systems, owing to their exceptional performance [3]. Many API recommendation systems focus primarily on direct or higher-order invocation history but ignore the textual information contained in the data [4,5]. This oversight prevents the models from capturing deeper relationships between nodes. To address this issue, certain approaches have been proposed [6–11]. These approaches either employ invocation or motif adjacency matrices to gather information about higher-order neighbors, or leverage textual attributes extracted by large language models (LLMs) in GNNs.

These GNNs, which rely on neighbor relationships, can all be classified as message passing neural networks (MPNNs), leading to a problem of neglecting the unique characteristics of different neighboring nodes or failing to capture long-distance dependencies [12]. To overcome this limitation, there is a growing trend towards using a global attention mechanism, also known as graph Transformer architectures, in graph data processing.

However, the graph Transformer architecture still faces some challenges such as high computational complexity and an inability to accurately represent the relationships between nodes [13]. Therefore, to design an outstanding Web API recommender system, this paper integrates semantic information and hierarchical positional relationships within the graph Transformer architecture. This approach aims to optimize time complexity and enhance the expression of inter-node relationships. During the design phase, our primary focus is on the following aspects:

- **Linearizing graph Transformer.** MPNNs are inherently limited by the topology of graphs, which makes it necessary to employ a graph Transformer to capture long-distance dependencies effectively. Moreover, given that the number of nodes in a graph is often substantial, leading to extreme computation times [14, 15]. It is also critical to design a model that achieves linear time complexity while leveraging relationships between nodes in the context of API recommendations to ensure efficiency and accuracy.

- **Comprehensive positional encoding.** Transformer architectures face challenges in handling non-Euclidean data, such as graphs, where nodes cannot simply be converted into sequences like words [16]. Therefore, information encoding is necessary to represent the nodes for subsequent tasks. Several studies have proposed positional encoding (PE) methods, but these typically consider only the semantic or positional information of the nodes [17–19], resulting in some loss of information. Consequently, it is essential to devise a strategy that integrates diverse information into PE in a comprehensive and discriminative manner, based on the combination of different relationships.
- **Higher-order relationships modeling.** Conventional multi-hop relationships used in many approaches may result in erroneous connections or fail to fully leverage associations when dealing with sparse data [9]. Therefore, it is essential to accurately identify higher-order relationships, as these are crucial for the comprehensive synthesis of information [20].
- **Semantic relationships modeling.** Many datasets contain overlooked textual information, which plays a significant role in improving the model's understanding of potential relationships with nodes. While some models have employed LLMs to encode these texts for downstream tasks [11,21,22], the richness of textual information utilized is not universally available across all datasets. Therefore, there is a necessity to develop an encoding method that enables efficient utilization of small amounts of text.

Based on the above analysis, in this paper, we propose a novel semantic and positional encoding method using LLM and motifs, and a linearizing graph Transformer for Web API recommendation. To extract higher-order relationships, we use motifs [23], which are substructures frequently found in complex networks and exhibit higher-order connectivity, as crucial tools for this purpose. Their important properties are demonstrated in Fig. 1. To extract the semantic information of the nodes, we utilize an LLM and employ various explicit and implicit fine-tuning strategies that can be implemented on a small amount of text to enhance its understanding of the inter-node information. To extract positional information, unlike others, we design a hierarchical method by incorporating spectral graph theory and the higher-order information provided by motifs. Additionally, we design a method to obtain the comprehensive PE by fusing semantic and location information. To reduce time complexity, we develop a linearizing graph transformer that leverages PE with rich positional information. This allows the model to use motifs to aggregate node information by distinguishing the importance of different nodes in API recommendation scenarios. Finally, we use a multi-layer perceptron (MLP) for the final optimization and recommendation. The contributions of our work are summarized as follows:

- We propose a PE method capable of capturing both semantic and higher-order position relationships from motifs, which helps the model to distinguish between nodes, which allows it to better fit the API recommendation.

- We propose a LLM-driven and motif-informed linearizing graph Transformer, named L-MTAR, for Web API recommendation. To the best of our knowledge, this is the first linearizing graph Transformer that incorporates motifs and semantic information for hierarchical PE in the context of recommendation.
- We propose a linearizing self-attention mechanism for graph Transformer that leverages activation function and re-weight mechanism. Unlike other methods, we optimize the attention coefficient of a node based on its higher-order information.
- We conducted comprehensive empirical studies using real-world datasets to assess the performance of L-MTAR. The results demonstrate the superiority of our model compared to some state-of-the-art models.

The remainder of this paper is organized as follows. Section 2 presents the related work; Section 3 gives the problem statement and definitions of main terminologies; Section 4 describes the PE method and presents the L-MTAR; Section 5 presents the experiment results and further analysis, and Section 6 concludes the paper.

## 2. Related work

In this section, we review the three research directions: MPNNs for recommendation, LLMs on graphs, graph PE, and graph Transformer architecture.

### 2.1. Message passing neural network for API recommendation

Most traditional API recommendation methods are based on collaborative filtering or MLPs [24]. However, as research continues to advance, MPNNs have demonstrated their superior performance. MPNNs represent a strategy for aggregating and updating information [25], where the embedding of a node is derived from its neighbors through aggregation. This strategy is particularly well-suited for Web API recommendation, as nodes can learn recommendations by considering their invocation history. Traditional methods of information propagation, which are constrained by the edges in the graph, can all be categorized under this strategy [26]. Liu et al. proposed RecRank, a graph-path API recommendation system that uses features based on usage paths to rank APIs [27]. Many approaches propagate messages directly on the original invoked bipartite graphs [28,29]. While these methods perform the required task effectively, they still need optimization due to issues such as data sparsity and over-smoothing, which can affect the recommendation results.

To address these challenges, some approaches have investigated higher-order information in graphs to improve the effectiveness of information aggregation [30,31]. Wang et al. have integrated motifs and graph attention mechanisms to hierarchically optimize their higher-order relationships and node embeddings [8]. While their approach effectively captures higher-order relationships between nodes, the MPNN architecture still limits the computation of long-range dependencies between nodes.

### 2.2. LLMs on graphs

In recent years, LLMs [32,33] have garnered significant attention for their exceptional capabilities across various NLP tasks [34]. They have also sparked interest among researchers studying their application to graph data [35]. Evaluations such as those by [36,37] demonstrate that LLMs show promise in understanding node relationships within graphs. However, relying solely on LLMs without explicit connections between nodes can limit the model's comprehension of graph data [38].

To tackle this issue, some approaches combine graph data with natural language processing techniques to improve the performance of the model on specific tasks. Tang et al. introduced GraphGPT, which integrates LLMs with graph structural knowledge through graph

instruction tuning [21]. This integration enhances its capability to capture higher-order information in graphs. Fatemi et al. use a graph structure to create prompts but crucial node invocation history essential for recommender systems is not further utilized [39].

### 2.3. Graph positional encoding

PE in the graph Transformer architecture is a crucial concept, particularly since nodes cannot be naturally sorted as in natural language processing. Many methods compose better PEs by combining graph structures. Zhang et al. proposed node substructure-based and position-based encoding methods [17]. Other works, such as [13,18], utilize spectral graph theory to encode nodes, but the consideration of local and higher-order information could still be enhanced.

Chen et al. employed positive definite kernels on graphs to obtain relative PE [40]. The structure-aware Transformer proposed by Chen et al. incorporates structural information into the original self-attention by extracting subgraphs [41]. However, this approach has high memory and time requirements.

### 2.4. Linearizing transformer

Transformer is an extensively studied architecture that has demonstrated highly competitive efficacy. However, it fixes the time complexity by constraining the order of computation due to the softmax function. Substantial efforts have been dedicated to exploring alternative methods to replace softmax. Some approaches have utilized physical models to optimize the process. For instance, Wu et al. employ the flow network perspective to reanalyze the information flow in the attention mechanism and introduce competition into the attention mechanism [15]. Additionally, Wu et al. attempt to establish a connection between diffusion differential equations and message passing, achieving a linearizing graph Transformer [42]. Since they are not intended for recommendation purposes only, the interrelationships between nodes in a recommendation scenario are not well considered.

In analyzing softmax, Qin et al. summarize the reasons for its effectiveness and alter the order of matrix calculation through approximation [43].

## 3. Definitions and problem statement

In this section, we will give definitions of the relevant knowledge and introduce the problem statement.

**Definition 1 (Mashup-API Invocation Graph).** We define a mashup-API invocation graph as  $IG = (U, V, E)$ ,  $U$  and  $V$  are sets of nodes in a graph, where  $U = \{u_1, u_2, \dots, u_{|U|}\}$  is a set of nodes representing mashups, and  $V = \{v_1, v_2, \dots, v_{|V|}\}$  is a set of nodes representing Web APIs, where  $|U|, |V|$  are the numbers of mashups and APIs. A set of oriented edges  $E = \{e_1, e_2, \dots, e_{|E|}\}$  where each edge starts from a node in  $U$  and ends at a node in  $V$ ,  $|E|$  is the number of edges. If there exists an  $e \in E$  between  $u \in U$  and  $v \in V$ , it means that  $u$  invoked  $v$ .

**Definition 2 (Network Motif).** The set  $\mathcal{M}$  contains network motifs  $M_i$ , which are the most common higher-order structures that occur frequently in a graph. There are usually more complex connections between the nodes that constitute the motifs. A network motif in bipartite graphs is defined as  $M_i = \{U_i \in U, V_i \in V, E_i \in E\}$ .

**Definition 3 (Motif Adjacency Matrix).** Given an  $IG$  and a type of motif  $M_i$  in which we are interested, we can define the motif adjacency matrix as  $A_i$ . For a value of  $k = 1$  at position  $(i, j)$  in  $A_i$ , it indicates that there are some instances of motif  $M_i$  between node  $i$  and node  $j$ .

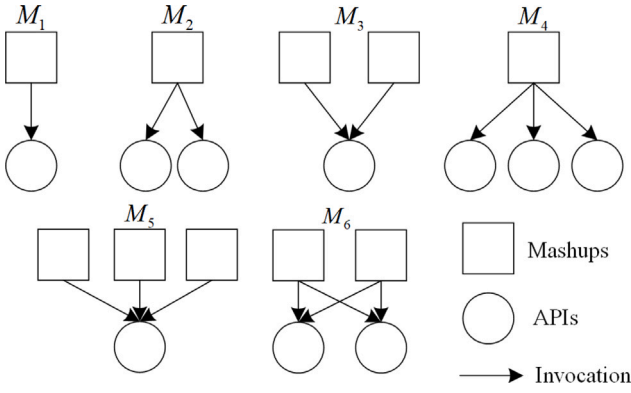


Fig. 2. The six motifs we used in this paper.

Table 1

Notations.

| Notation                | Explanation                                                    |
|-------------------------|----------------------------------------------------------------|
| $IG = (U, V, E)$        | Mashup-API invocation graph (abbreviated as $IG$ )             |
| $\mathcal{M} = \{M_t\}$ | The set of $\mathcal{T}$ motifs $t = 1, 2, \dots, \mathcal{T}$ |
| $\mathcal{A} = \{A_t\}$ | The set of $\mathcal{T}$ motif adjacency matrices              |
| $\mathcal{N}_i^r$       | The set of node $i$ 's neighbors in $A_t$                      |
| $D_i$                   | The textual information of node $i$                            |
| $emb_i$                 | The semantic embedding of node $i$                             |
| $L^t$                   | The symmetric normalized Laplacian of $A_t$                    |
| $\lambda_z^t$           | The $z$ th eigenvalues of $L^t$ (ascending order)              |
| $\phi_z^t$              | The eigenvector of $\lambda_z^t$                               |
| $f_i^{L_t}$             | The global positional encoding of node $i$ in $L_t$            |
| $f_i$                   | The global position encoding of node $i$                       |
| $P_i$                   | The projected $f_i$                                            |
| $e_{i,j}$               | The importance of node $j$ to node $i$ in $all_{neighbors}$    |
| $a_{i,j}$               | The weight coefficient calculated from $e_{i,j}$               |
| $Q, K, V$               | The query, key, and value matrices                             |
| $\gamma_i$              | The importance of node $i$ in re-weight mechanism              |
| $O$                     | The final output matrix                                        |

**Definition 4 (Node's Higher-order Neighbors).** We define a set of node  $i$ 's higher-order neighbors in  $A_t$  as  $\mathcal{N}_i^t$ . Given a  $A_t$ , for a value  $k$  at position  $(i, j)$  in the matrix.  $k = 1$  implies that node  $i$  and node  $j$  are neighbors of each other.

**Definition 5 (Node-level Textual Information).** We define a Node-level textual information as  $D_i$ . In this paper,  $D_i$  is textual information about the category and the neighbors' ID of node  $i$ .

**Problem Statement:** Given an  $IG g = (U, V, E)$ , a mashup  $u_i$  and a set of APIs  $V_{u_i}$  which  $u_i$  has invoked, our goal is to recommend the top  $K$  APIs from  $V - V_{u_i}$  that should be most likely to be invoked by  $u_i$ .

In this paper, we consider motifs consisting of up to four nodes in bipartite graphs, which are six motifs in total. The six motifs are shown in Fig. 2. The main notations used in this paper are summarized in Table 1.

#### 4. Methodology

In this section, we will discuss the detailed design of our model. Fig. 3 depicts the architecture of the model. The L-MTAR is designed using a four-layer structure: (1) the LLM-based semantic encoding. (2) the motif-based positional encoding module for obtaining structural information about nodes from different levels. (3) the linearizing graph Transformer for aggregating information from the entire graph. (4) the MLP module for optimizing node embeddings to better recommendations.

##### 4.1. LLM-enhanced semantic encoding

GNNs achieve information understanding by recursively exchanging and aggregating information between graph nodes. However, there exists additional information beyond spatial relationships, such as textual content. Inspired by LLMs, our objective is to create a graph-oriented semantic encoding method that produces high-quality embeddings, even in the absence of rich textual data commonly found in knowledge graphs.

In Web API recommendation, the textual information associated with APIs and mashups often contains intricate relationships. However, directly encoding this information with LLMs typically focuses solely on patterns learned from the pre-trained corpus [19]. To enhance the LLM's semantic understanding in API recommendation scenarios, we devised four fine-tuning methods inspired by [44], aiming for explicit and implicit semantic alignment, as shown in Fig. 4.

The explicit methods enable the language model to directly learn the correspondence between nodes and texts. In contrast, the implicit methods account for collaborative relationships among nodes, recognizing that mashups may involve multiple types of APIs. Learning these collaborative relationships is crucial for effective recommendations. We argue that the vast majority of valid relationships are captured through explicit methods, whereas implicit methods aim to extract information that is often overlooked. Consequently, there are more explicit fine-tuning approaches than implicit ones. Additionally, we choose to use only category information because numerous techniques already exist for leveraging large volumes of text to obtain semantic information. Inevitably, some datasets lack such rich textual content, so by relying on category information, we aim to develop multiple fine-tuning methods that allow the LLM to apply to a broader range of data.

In our experiments, we utilize GLM3, the third generation of the open source LLM [45], as the base model. We employ LoRA [46] for fine-tuning, resulting in a refined model denoted as  $GLM_{rec}$ .

Based on the enhanced understanding of textual mutual information among nodes by  $GLM_{rec}$ , we propose textual information for nodes illustrated in Fig. 5. This representation takes the form of  $D_i = [node's\ category] : \{< neighbors' ID >, \}$ .

The rationale for excluding the nodes' ID is that two nodes may have adjacent IDs but lack adjacent semantic relationships because the ID only relates to the order in which the data was obtained. Moreover, including the neighbors' category in the textual representation could lead to repetitive words, hindering the model's generalization ability.

$$emb'_i = GLM_{rec}(D_i)$$

$$emb'_i = \frac{1}{s+1} \left( \sum_{j=0}^{s-1} emb''_i[j] \right) + emb''_i[Len_i] \quad (1)$$

where  $emb''_i \in \mathbb{R}^{Len_i \times 4096}$ ,  $Len_i$  is the length of  $D_i$ 's tokens after the tokenizer,  $j \in [0, (s-1)]$  are the corresponding tokens' position of the text of  $node's\ category$  after the tokenizer,  $emb''_i[j]$  represents the  $j$ th row of the  $emb''_i$ . We select the embedding of the last token, denoted as  $emb''_i[Len_i]$ , to represent the embedded representation that integrates all token information because the vast majority of prevailing models employ only causal attention mechanisms, i.e., they are based on decoder-only architectures. Here, 4096 represents the token embedding dimension of the  $GLM_{rec}$  model. Then, we average these embeddings to derive  $emb'_i$ . This averaged embedding incorporates both the node's type and neighbor information.

Since there is a gap between semantic and graph structure information, direct concatenate could disrupt the model's functionality. In contrast, LLMs have numerous dimensions to enhance their generative capacity. Therefore, we employ UMAP [47] and MLP to reduce the dimensions of  $emb'_i$  and align it with the graph structure, forming the final semantic embedding  $emb_i \in \mathbb{R}^Q$  as shown in Eq. (2). For each layer of the MLP, we utilize a weight matrix  $W_r \in \mathbb{R}^{F_i \times F_o}$ , where  $F_i$



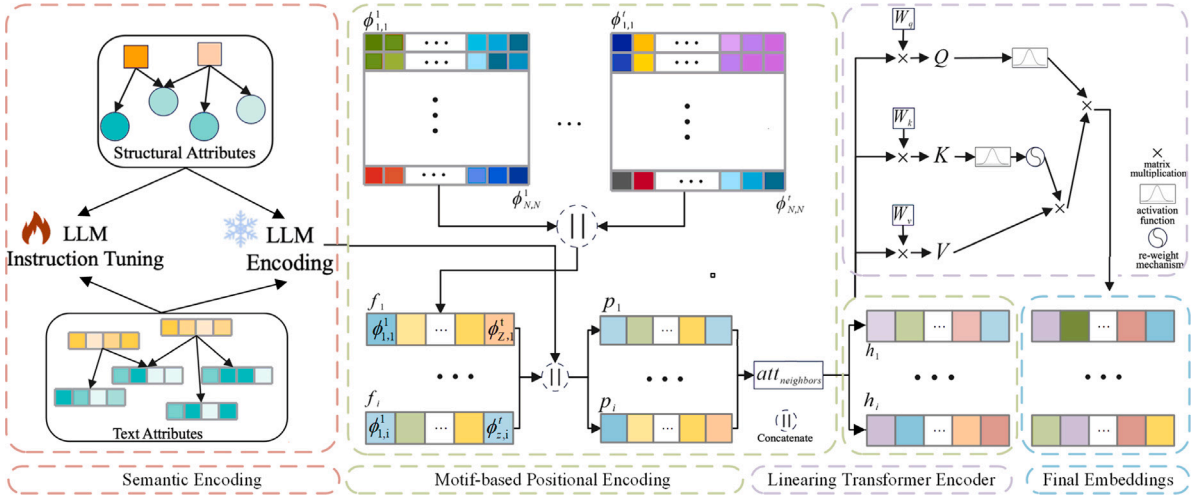


Fig. 3. The modules of the proposed L-MTAR.

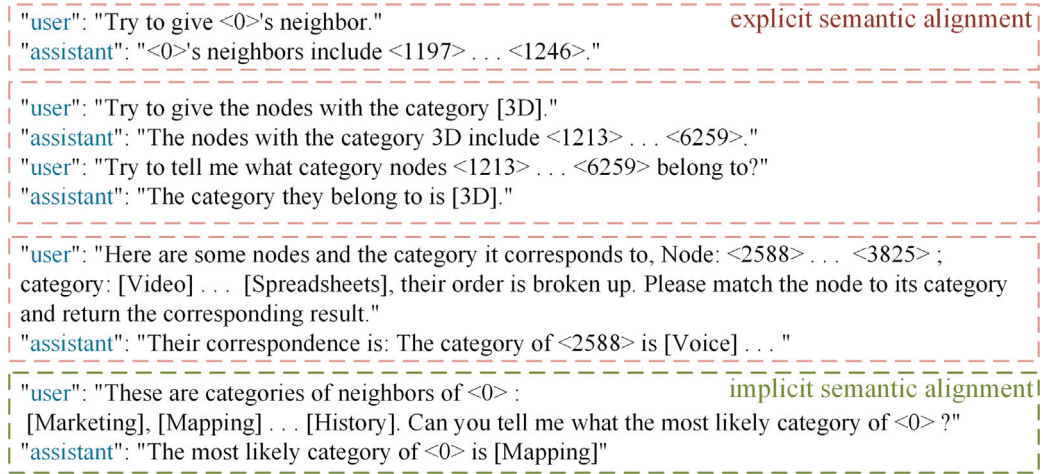


Fig. 4. The examples of four instruction designs.

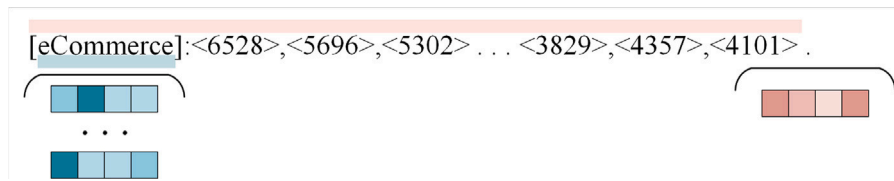


Fig. 5. The example of a textual representation of a node.

denotes the dimension of the previous layer's output and  $F_o$  denotes the dimension of the current layer's output, which must be defined.

$$\begin{aligned} emb_i &= MLP(UMAP(emb'_i)) \\ &= UMAP(emb'_i)W_r \end{aligned} \quad (2)$$

#### 4.2. Motif-based positional encoding

In the Transformer, one of the most basic concepts is the use of sine and cosine functions as PE functions for sequences. However, for graphs, direct application of these methods is not feasible, due to the absence of sequential relations between nodes. Simply generalizing the nodes in the graph to sequences would result in the loss of significant structural information. To retain as much structural information as possible, it is essential to find a suitable alternative method.

##### 4.2.1. Global positional encoding

The eigenvectors of the Laplacian of a graph serve as natural equivalents to PE [13]. They can be employed to differentiate between various graph structures, as their values reflect the magnitude of the resonance frequency [48]. Previous research has shown that utilizing spectral information expressed by Laplacian is an effective method for preserving the global features of nodes [18,49]. Moreover, the use of motif adjacency matrices assists the model in identifying potential relations between nodes that might be easily overlooked [9]. Therefore, in obtaining the global PE, we opt for the use of motif adjacency matrices instead of the direct invocation matrix. We refer to the algorithm in [8] to obtain the motif adjacency matrices, which have a time complexity of  $O(|E|^2)$  and can therefore be solved in finite time.

We define the Laplacian of  $A$ , as  $L^t$ , the  $t$ th eigenvalues of  $L^t$  as  $\lambda_z^t$  (ascending order), and the eigenvector corresponding to  $\lambda_z^t$  as  $\phi_z^t$ . The

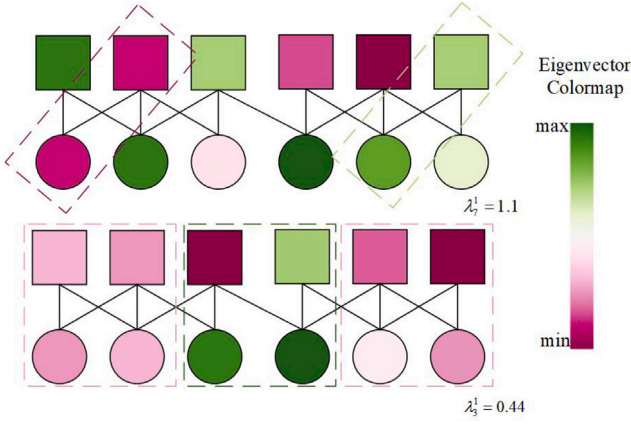


Fig. 6. Examples of eigenvalues  $\lambda_z^1$  and eigenvectors  $\phi_z^1$  of a simple  $L_1$ .  $\phi_{z,i}^1$  is the value of the  $i$ th position in the  $\phi_z^1$  and also corresponds to node  $i$ . The color of a node represents the value of  $\phi_{z,i}^1$  and the value of  $\lambda_z^1$  corresponds to the frequency. The eigenvector corresponding to the low-frequency  $\lambda_1^1$  is uniformly distributed throughout the substructure of the graph, while the eigenvector corresponding to the high-frequency  $\lambda_7^1$  usually resonates in very localized structures (e.g., a pair of adjacent nodes).

role of eigenvectors for a simple IG is presented in Fig. 6.

$$L' = I - D^{-\frac{1}{2}} A_l D^{-\frac{1}{2}} \quad (3)$$

where  $L' \in \mathbb{R}^{N \times N}$  is a symmetric normalized Laplacian,  $I$  is an identity matrix and  $D$  is the diagonal degree matrix with  $i$ th diagonal entry  $D_{i,i} = \deg_i = \sum_{j \in \mathcal{N}_i^+} 1$  of  $L'$ , they have the same dimensions as  $L'$ . As mentioned in Fig. 6, the larger eigenvalue  $\lambda_z^1$  represents larger resonance frequency, and the corresponding eigenvector  $\phi_z^1$  records the relative localized structures between nodes. Instead, as we need, relatively global structures are recorded. Therefore we use the  $\phi_z^1$  corresponding to the  $Z$  smallest  $\lambda_z^1$  to obtain global structural information about the nodes. Given an  $A_l$ , we define the global PE set of the nodes as  $I_l = \{f_1^{L_l}, f_2^{L_l}, \dots, f_N^{L_l}\}$ .  $f_i^{L_l} \in \mathbb{R}^Z$  computed by (4) is the row vector and the global PE of the node  $i$  in  $A_l$ ,  $N = |U + V|$  is the number of all nodes and  $Z$  is the number of dimensions.

$$f_i^{L_l} = \parallel_{z=1}^Z (\phi_{z,i}^1) \quad (4)$$

where  $\phi_{z,i}^1$  is the value of the  $i$  position in the  $\phi_z^1$ ,  $\parallel$  represents the concatenation operation and  $i$  corresponds to the index of the node. We concatenate the values from the same positions of the  $Z$  eigenvectors  $\phi_z^1$  to obtain a higher-order global PE obtained by a motif  $M_l$ .

To obtain a more comprehensive information, we utilize  $\mathcal{T}$  different  $L_l$  to derive  $f_i^{L_l}$ . Subsequently, in (5), we concatenate these individual features to form the higher-order global PE set  $I_{glb} = \{f_1', f_2', \dots, f_N'\}$ , where  $f_i' \in \mathbb{R}^{F'}$  and  $F' = \mathcal{T}Z$  represents the dimension of  $f_i'$ .

$$f_i' = \parallel_{l=1}^{\mathcal{T}} (f_i^{L_l}) \quad (5)$$

Finally, we concatenate the semantic encoding  $emb_i$  with the higher-order global PE  $f_i'$  to form a global PE  $f_i \in \mathbb{R}^{F'+Q}$  that contains semantics.

$$f_i = f_i' \parallel emb_i \quad (6)$$

#### 4.2.2. Local positional encoding

Above we have examined the relations between nodes from a global perspective, highlighting the overall context of the graph. It aims to capture global attributes and long-range dependencies of nodes, aspects that are not typically addressed in general MPNNs.

Nevertheless, local relations remain crucial in Web API recommender systems, enabling us to focus on the connections between

nodes and their immediate neighbors. This is significant for aggregating information about historical interactions between nodes and thereby determining their direct similarity. In this section, we will employ global PE and integrate it with local structures to create the ultimate node PE.

Inspired by GAT [50], we use a self-attention network to aggregate local information between nodes. First, through the  $I_{glb}$ , the linear transformation layers produce a new set of nodes' features  $P = \{p_1, p_2, \dots, p_N\}$ ,  $p \in \mathbb{R}^F$  by a weight matrix  $W_I \in \mathbb{R}^{(F'+Q) \times F}$ , where  $F$  is the dimension of the projected embeddings.

$$p_i = f_i W_I \quad (7)$$

Then, the attention coefficient  $e_{i,j}$  that can represent the importance of node  $j$  for node  $i$  is computed by a deep neural network  $att_{neighbors} : \mathbb{R}^F \times \mathbb{R}^F \rightarrow \mathbb{R}$ . After the global PE completes the transformation, we use a single-layer feedforward neural network containing only the parameter  $a \in \mathbb{R}^{2F}$  to obtain the attention coefficients.

$$\begin{aligned} e_{i,j} &= att_{neighbors}(p_i, p_j) \\ &= \text{LeakyReLU}([f_i W_I \parallel f_j W_I](a)^T) \end{aligned} \quad (8)$$

where  $\cdot^T$  is the transpose operation and  $\parallel$  is the concatenate operation.

In this step, we consider only the local relations of the nodes. Therefore we capture node  $j \in \mathcal{N}_i^+$ ,  $\tau = \min(t)$  as the basis for getting the final PE of node  $i$ . The reason for considering only neighbors in  $\mathcal{N}_i^+$  is we believe that a relatively simple motif contains more direct relations between nodes, thus directly reflecting and enhancing the dependencies between nodes in the construction of PE.

After that, we use the softmax function to normalize the  $e_{i,j}$  to get the final weight coefficient  $\alpha_{i,j}$ , which in turn allows the features to be evenly aggregated among nodes.

$$\alpha_{i,j} = \text{softmax}(e_{i,j}) = \frac{\exp(e_{i,j})}{\sum_{x \in \mathcal{N}_i^+} \exp(e_{i,x})} \quad (9)$$

Finally, we use the computed  $\alpha_{i,j}$  to aggregate the features between neighbors.

$$h_i = \sum_{j \in \mathcal{N}_i^+} \alpha_{i,j} \cdot p_j \quad (10)$$

To improve the generalization performance, we improve (10) by fusing more information using a multi-head mechanism.

$$h_i = \sum_{g'=1}^{G'} \sum_{j \in \mathcal{N}_i^+} \alpha_{i,j}^{g'} \cdot p_j^{g'} \quad (11)$$

After completing the above steps, We stack  $h_i$  as a row vector to get the final PE matrix  $I_o = [h_1, h_2, \dots, h_N]^T$ ,  $I_o \in \mathbb{R}^{N \times F}$ , which not only contains the global and local relations between the nodes, but also enhances the neglected information in the direct adjacent matrix by the higher-order relations.

#### 4.3. Motif-based linearizing transformer

Having acquired  $I_o$  that incorporates hierarchical positional information, we possess the foundation for merging finer details among nodes within a graph Transformer. Its attention mechanism, unconstrained by the limitations imposed by neighbor relations among nodes, not only captures long-distance dependencies between nodes but also mitigates the over-smoothing issue arising from multiple message propagation.

The dot product attention mechanism, employing Softmax normalization, serves as the pivotal component in Transformers for calculating correlation. Nonetheless, its computational overhead becomes substantial, particularly in scenarios involving a large number of nodes within the graph, due to its quadratic spatial and temporal complexity concerning the node's number. Certain methodologies address this issue by altering the sequence of matrix multiplication and approximating

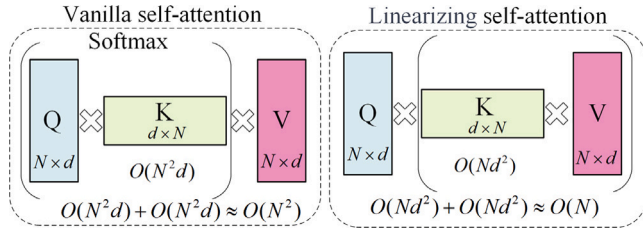


Fig. 7. Illustration of the computation of the Vanilla and linearizing self-attention. We will analyze this in detail in 4.5.

the impact of Softmax [42,43], as depicted in Fig. 7. We were inspired by their alteration in the order of matrix computation. Consequently, we specifically tailored our approach to consider the significance of the numerous higher-order connections between nodes that they overlook.

Softmax exhibits two critical properties: (1) all values in the attention matrix are non-negative; (2) it furnishes a non-linear mapping, rendering the attention distribution smoother and ensuring stable training [43]. To approximate the softmax effect and leverage the inter-node correlations conveyed by motifs, we have devised a linearizing graph Transformer, which can be formulated as (12). It comprises two key components: (1) the activation function  $\sigma(\cdot)$ ; (2) a motif-based re-weighting mechanism  $S(\cdot, \cdot)$ .

$$O = S(\sigma(Q), \sigma(K))V = S(Q', K')V \quad (12)$$

$$\begin{aligned} Q &= I_o W_q \\ K &= I_o W_k \\ V &= I_o W_v \end{aligned} \quad (13)$$

where  $Q, K, V, O \in \mathbb{R}^{N \times d}$  represents the query, key, and value matrix, respectively.  $W_q, W_k, W_v \in \mathbb{R}^{F \times d}$  denote the weight matrices associated with the  $Q, K, V$ , and  $O$  is the output matrix.

#### 4.3.1. Activation function $\sigma(\cdot)$

To ensure the attainment of an entirely non-negative attention matrix, the  $\sigma$  in L-MTAR is expressed as follows:

$$\sigma(\cdot) = \text{SELU}(\cdot) + 2 \quad (14)$$

$Q$  and  $K$  are converted to  $Q'$  and  $K'$  by  $\sigma(\cdot)$ . The SELU is the modified ELU that allows the data to move closer to the standard normal distribution. Also, the minimum value after activation is close to  $-2$ , the result is added by 2 to ensure non-negativity.

#### 4.3.2. Motif-based re-weight mechanism $S(\cdot, \cdot)$

After converting the matrices to a non-negative form, it becomes imperative to devise a method for optimizing the attention coefficients to focus on the connections between the nodes. This process generalizes to the assumption that the node with the most higher-order neighbors has the highest significance which is skillfully linked to the motifs that contain related nodes, thereby discerning the significance of that particular node to others.

In  $A_i$ , the number of motifs  $M_i$  that node  $i$  can form with other nodes is denoted by  $\gamma_i'$ . A larger  $\gamma_i'$  implies a greater number of associations that we attribute to node  $i$  with other nodes. Therefore, the features of node  $i$  are considered more important and should be noticed by other nodes during the information aggregation process. Relatively, if  $\gamma_i'$  is relatively small, the node  $i$  is unimportant. For a more comprehensive analysis of inter-node correlations, considering all motifs we are interested in, the importance of node  $i$  is represented as  $\gamma_i'$ :

$$\begin{aligned} \gamma_i &= \sum_{t=1}^{\tau} \gamma_i^t = \sum_{t=1}^{\tau} \sum_{j=1}^N A_{t(i,j)} \\ \gamma_i' &= \text{norm}(\gamma_i) = \frac{\gamma_i - \min(\gamma)}{\max(\gamma) - \min(\gamma)} \end{aligned} \quad (15)$$

where  $A_{t(i,j)}$  represents the value at position  $(i, j)$  of the  $A_t$ . Furthermore,  $\min(\gamma)$  corresponds to the minimum value among all nodes'  $\gamma_i$ , and  $\max(\gamma)$  is its counterpart representing the maximum value. Notably, due to potential unevenness in the number of motifs associated with a node, normalization is performed. Given this definitions, the expression for  $S(\cdot, \cdot)$  can be articulated as:

$$\begin{aligned} S(Q', K') &= \frac{Q'_i K'_j{}^T \theta(\gamma_j')}{\sum_{x=1}^N Q'_i K'_x{}^T \theta(\gamma_x')} \\ &= \frac{Q'_i K'_j{}^T \sqrt{1 - (1 - \gamma_j')^2}}{\sum_{x=1}^N Q'_i K'_x{}^T \sqrt{1 - (1 - \gamma_x')^2}} \end{aligned} \quad (16)$$

where the function  $\theta(\cdot) = \sqrt{1 - (1 - \cdot)^2}$  serves as a non-linear re-weighting function that is used to dynamically adjust the coefficients obtained directly from matrix multiplication, and  $\gamma_j'$  as the independent variable. It can be demonstrated that this function is Concave. Given that  $\gamma_j' \in [0, 1]$ , it follows that  $\theta(\gamma_j') \in [0, 1]$ . Consequently, this function is designed to focus nodes with a small number of motif instances, effectively compressing their attention weights, while maintaining the original attention weights for nodes with a larger number of instances.

Finally, by combining (12) and (16), the nodes' final embedding  $O_i$  produced by L-MTAR can be expressed as:

$$\begin{aligned} O_i &= \sum_{j=1}^N S(Q'_i, K'_j) V_j \\ &= \frac{\sum_{j=1}^N Q'_i K'_j{}^T \theta(\gamma_j') V_j}{\sum_{j=1}^N Q'_i K'_j{}^T \theta(\gamma_j')} \\ &= \frac{\sum_{j=1}^N Q'_i K'_j{}^T \sqrt{1 - (1 - \gamma_j')^2} V_j}{\sum_{j=1}^N Q'_i K'_j{}^T \sqrt{1 - (1 - \gamma_j')^2}} \end{aligned} \quad (17)$$

The final output is obtained by a multi-head mechanism.

$$O_i = \parallel \sum_{g=1}^G S(Q_i^g, K_j^g) V_j^g \quad (18)$$

Subsequently, given the absence of constraint on the exponential operation, the attention operation for the multiplication of three matrices can be acquired by altering the order of computation. Without loss of generality, it can be denoted as:

$$(Q' K'^T) V = Q' (K'^T V) \quad (19)$$

#### 4.4. Multi-layer perceptron for recommendation

This section aims to optimize  $O_i$  by using an MLP and nonlinear transformations to generate recommendation results  $\hat{R}_{m,a}$ . We employ two MLPs for the nodes of both mashups and APIs separately. Finally, we apply ELU for activation.

$$b_i = \text{ELU}(O_i W_r) \quad (20)$$

In our experiment, we adopt a three-layer MLP to optimize features and use the dot product of  $b_m$  and  $b_a$  to predict their correlation  $\hat{R}_{m,a}$ .  $b_m$  and  $b_a$  are the final representations of mashups and APIs.

$$\hat{R}_{m,a} = b_m \odot b_a \quad (21)$$

After softmax, the value of  $\hat{R}_{m,a} \in [0, 1]$  reflects the likelihood of  $m \in U$  invokes  $a \in V$ .

A value of 1 is assigned to an existing invocation, and 0 is assigned otherwise. Our model is trained in an end-to-end manner, with the parameters of the entire model optimized by a binary cross-entropy loss function  $\mathcal{L}$ .

$$\begin{aligned} \mathcal{L} &= -\frac{1}{N_t} \sum_{m \in U, a \in V} R_{m,a} \log \hat{R}_{m,a} \\ &\quad + (1 - R_{m,a}) \log (1 - \hat{R}_{m,a}) \end{aligned} \quad (22)$$

where  $R_{m,a}$  is the actual value between  $m$  and  $a$ , and  $N_i$  is the number of all node pairs which we use to compute loss.

#### 4.5. Complexity analysis

Since our method consists of two modules, it increases the overall complexity. In practice, we can use the algorithm in [8,13] to find the motif adjacency matrices and eigenvectors. Their time complexity are  $O(|E|^2)$  and  $O(Z|E|^\tau)$ , respectively.  $|E|^\tau$  denotes the total number of edges in  $A_\tau$ . However, conceptually, global PE is similar to text embedding, an independent pre-training process that can be replaced and should be considered separate from the models' complexity. Hence, we analyze all modules containing neural networks and demonstrate that our graph Transformer achieves linear complexity. Our analysis assumes that all models under consideration consist of only a single layer and a single attention head.

In the graph Transformer module, the dimensions of hidden features and input features are represented by  $d$  and  $F$ , and the total number of parameters is  $3Fd$  which includes three weight matrices, a consistent characteristic across all Transformer architectures. For the  $att_{neighbors}$ , crucial for obtaining local PE, the number of parameters involved is  $(F' + Q)F + 2F$ , where  $(F' + Q)F$  denotes the parameters of the weight matrix, and  $2F$  accounts for the parameters in  $att_{neighbors}$ .

Therefore, the overall number of parameters is  $(F' + Q)F + 2(F' + Q) + 3Fd$ . In comparison to traditional graph Transformer architecture, there is only a slight increase attributed to the incorporation of the graph attention mechanism. However, when contrasted with the parameters in H-MGSR [9], denoted as  $\mathcal{T}(NF + 2F) + F + N$ , which similarly utilizes multiple motif adjacency matrices for attention computation, our model exhibits a reduction in the number of parameters.

For time complexity, disregarding operations related to the weight matrices that are common across all models can be ignored, the unimproved graph Transformers exhibits a time complexity of  $O(N^2d)$ . This arises from performing two matrix multiplications involving  $\mathbb{R}^{N \times d}$  with  $\mathbb{R}^{d \times N}$ , and  $\mathbb{R}^{N \times N}$  with  $\mathbb{R}^{N \times d}$ . After incorporating the activation function and re-weight mechanism instead of a softmax function, the order of matrix computations can be rearranged to  $\mathbb{R}^{d \times N}$  with  $\mathbb{R}^{N \times d}$ , and  $\mathbb{R}^{N \times d}$  with  $\mathbb{R}^{d \times d}$ , resulting in time complexity of  $O(Nd^2)$ . Given that  $d \ll N$ , the two components can be separately simplified to  $O(N^2)$  and  $O(N)$ . Thereby, our Transformer implements linearizing complexity. The time complexity of the graph attention mechanism for obtaining PE can be expressed as  $O(|E|^\tau F)$ . Therefore, the overall time complexity is  $O(Nd^2 + |E|^\tau F)$ .

In conclusion, while our approach includes additional neural networks, resulting in a modest increase in resource consumption compared to linearizing models [15,42], it offers a more comprehensive and effective methodology for Web API recommendation.

## 5. Experiments

In this section, we will first introduce our datasets and describe the parameter settings of our model. Next, we will compare our model against existing methods that can be applied to Web API recommendations. Finally, we performed ablation experiments and further analysis.

### 5.1. Dataset and baselines

**ProgrammableWeb Dataset<sup>1</sup>:** This dataset contains 17,829 APIs and 6340 mashups. We then removed blank APIs and mashups, leaving 5691 mashups and 1170 APIs, which together contain 10,788 invoked pairs. The sparsity is 0.0016.

**MovieLens Dataset [51]:** This dataset contains 1,000,209 ratings from 6040 users on 3900 movies, and we keep only the highest-rated

**Table 2**

Hyperparameters.

| Hyperparameters                                    | Values                    |
|----------------------------------------------------|---------------------------|
| The number of dimensions in Semantic Encoding $Q$  | 256                       |
| The number of dimensions in global PE $F'$         | 1024                      |
| The number of hidden dimensions in local PE $F$    | 512                       |
| The number of dimensions in graph Transformer $d$  | 48                        |
| The number of multi-heads in graph Transformer $G$ | 8                         |
| The number of multi-heads in local PE $G'$         | 3                         |
| The learning rate                                  | 0.0015                    |
| The epoch of ProgrammableWeb dataset               | 1400                      |
| The motifs in ProgrammableWeb dataset              | $M_3, M_4, M_5$ and $M_6$ |
| The epoch of MovieLens dataset                     | 2200                      |
| The motifs in MovieLens dataset                    | $M_1$ and $M_6$           |

interactions as the dataset. We ended up with 186,213 interactions from 4977 users on 3182 movies. The sparsity is 0.0117. We use this dataset to test the generalization performance of our model at sparsity distinct from the web API dataset.

**Baselines:** We compared L-MTAR with some state-of-the-art methods:

- LightGCN [52]: a GCN-based network that is simpler and better suited for recommendations.
- NeuMF [53]: a combination of neural networks and matrix factorization for learning user-item invocations.
- DIFFormer [42]: a linearizing transformer inspired by energy constrained diffusion processes.
- Flowformer [15]: a linearizing transformer with conservation flows.
- GAT [50]: a graph attention network.
- H-MGSR [9]: a Web API recommender system by using hierarchical attention mechanism on motif adjacency matrices.

### 5.2. Training settings

The experiments are conducted on a workstation utilizing the NVIDIA A100 GPU with 40 GB memory. All datasets are split into a train set, validation set, and test set with ratios 60%, 20%, and 20% respectively. We use the validation set to make decisions regarding hyper-parameters. The source code is implemented based on PyTorch 2.1.1 with cuda 12.1. Using the validation set, we use the experience to optimize through multiple experiments to determine final hyper-parameter settings. We show the settings of the hyperparameters in Table 2.

### 5.3. Evaluation metrics

$$HR@K = \frac{\text{number of hits@K}}{N_m} \quad (23)$$

$$\begin{aligned} DCG_m@K &= \sum_{i=1}^K \frac{2^{R_{m,i}} - 1}{\log_2(i + 1)} \\ I DCG_m@K &= \sum_{j=1}^K \frac{2^{R_{m,j}} - 1}{\log_2(j + 1)} \\ N DCG_m@K &= \frac{DCG_m@K}{I DCG_m@K} \\ N DCG@K &= \frac{\sum_{m \in \{test\_set \cap M\}} N DCG_m@K}{|test\_set \cap M|} \end{aligned} \quad (24)$$

We use Hit Ratio (23) and Normalized Discounted Cumulative Gain (24) to evaluate the accuracy of our method in API recommendations.

We randomly select 60%, 20%, and 20% of the data in each dataset as training, testing, and validation sets, respectively. After the model furnishes its recommendations for each mashup, a sorting operation is performed based on  $\hat{R}_{m,a}$ . Following this, the top- $K$  ranked results

<sup>1</sup> We crawled [www.programmableweb.com](http://www.programmableweb.com) in October 2018.



**Table 3**

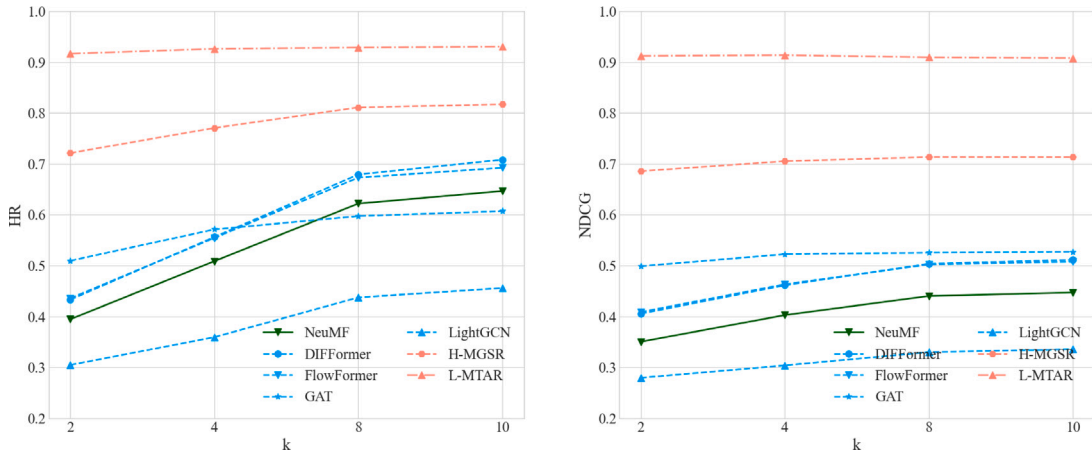
Comparison over different k on the ProgrammableWeb dataset.

| Model      | Metrics       |               |               |               |               |               |               |               |
|------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
| Name       | HR@2          | HR@4          | HR@8          | HR@10         | NDCG@2        | NDCG@4        | NDCG@8        | NDCG@10       |
| LightGCN   | 0.3049        | 0.3594        | 0.4376        | 0.4561        | 0.2796        | 0.3040        | 0.3301        | 0.3357        |
| NeuMF      | 0.3946        | 0.5088        | 0.6221        | 0.6467        | 0.3504        | 0.4028        | 0.4405        | 0.4472        |
| DIFFormer  | 0.4332        | 0.5562        | 0.6793        | 0.7083        | 0.4056        | 0.4615        | 0.5038        | 0.5117        |
| Flowformer | 0.4359        | 0.5545        | 0.6731        | 0.6924        | 0.4089        | 0.4634        | 0.5024        | 0.5080        |
| GAT        | 0.5097        | 0.5712        | 0.5975        | 0.6072        | 0.4990        | 0.5226        | 0.5256        | 0.5272        |
| H-MGSR     | 0.7214        | 0.7707        | 0.8111        | 0.8172        | 0.6858        | 0.7053        | 0.7135        | 0.7133        |
| Our models | <b>0.9165</b> | <b>0.9262</b> | <b>0.9288</b> | <b>0.9306</b> | <b>0.9123</b> | <b>0.9137</b> | <b>0.9096</b> | <b>0.9081</b> |

**Table 4**

Comparison over different k on the MovieLens dataset.

| Model      | Metrics       |               |               |               |               |               |               |               |
|------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
| Name       | HR@2          | HR@4          | HR@8          | HR@10         | NDCG@2        | NDCG@4        | NDCG@8        | NDCG@10       |
| LightGCN   | 0.3739        | 0.4925        | 0.5729        | 0.5970        | 0.3264        | 0.3763        | 0.3994        | 0.4035        |
| NeuMF      | 0.4503        | 0.593         | 0.7457        | 0.7829        | 0.3909        | 0.4543        | 0.5018        | 0.5102        |
| DIFFormer  | 0.3739        | 0.5562        | 0.7598        | 0.8010        | 0.3308        | 0.4173        | 0.4921        | 0.5064        |
| Flowformer | 0.3568        | 0.4563        | 0.6332        | 0.6714        | 0.3082        | 0.3576        | 0.4102        | 0.4151        |
| GAT        | 0.3940        | 0.5578        | 0.6894        | 0.7347        | 0.3343        | 0.4084        | 0.4526        | 0.4640        |
| H-MGSR     | 0.4925        | 0.6714        | 0.7920        | 0.8191        | 0.4379        | 0.5163        | 0.5495        | 0.5537        |
| Our models | <b>0.7246</b> | <b>0.8543</b> | <b>0.9296</b> | <b>0.9497</b> | <b>0.6530</b> | <b>0.7090</b> | <b>0.7266</b> | <b>0.7282</b> |

**Fig. 8.** HR and NDCG results in ProgrammableWeb dataset.

are collated to form the recommendation list, and metric calculations ensue.

For each mashup, the HR result is computed according to (23), where *number of hits@K* signifies the number of mashups for which the recommended API by our model is included in the set of it has invoked and  $N_m$  denotes the number of mashups in the test set. We use NDCG to measure the effectiveness of API ranking in the prediction list. NDCG is calculated by (24), for the normalization factor  $IDCG_m@K$ , we use the values of  $R_{m,i}$  to sort  $K$   $R_{m,i}$  in descending order to form  $R_{m,j}$ , to ensure that the results can be compared across different values of  $K$ .  $test\ set \cap M$  represents the mashups in the test set, and  $|test\ set \cap M|$  is the number of such mashups.

#### 5.4. Results comparison

For both the baseline models and L-MTAR, we ran the models until achieving optimal results in terms of HR and NDCG across various combinations. The initial node embedding dimensions for all models are consistent with L-MTAR. For DIFFormer, Flowformer, and GAT, we used the direct adjacency matrix to obtain the global PE as the initial embedding of the nodes to preserve positional information. The performance of the models is presented in Tables 3 and 4. Additionally, we employed line plots, shown in Figs. 8 and 9, to facilitate result comparison. The plots use color to classify models: the first group, in

green, represents techniques related to matrix decomposition; the second group, in blue, represents models using GNNs while disregarding higher-order structures; and the third group, in red, represents models incorporating higher-order structures.

Notably, in the ProgrammableWeb dataset, NeuMF, a model utilizing the decomposition and neural network, falls short in achieving results compared to most methods. Most of the models based on MPNN or graph Transformer architecture have achieved moderate performance. The model that takes into account higher-order structures to gain a better understanding of the hidden relationships between nodes outperforms the other models in almost all metrics. Our model attains the highest performance by simultaneously considering the higher-order and semantic relationships and employing an aggregating mechanism not confined to neighbors. Furthermore, by examining the slopes, we can identify variations in the performance of the same model at different  $K$ .

In another dataset, a similar scenario emerges where the model that takes into account higher-order structures exhibits superior performance. We believe this dataset, being relatively dense, facilitates effective performance by matrix decomposition methods. Thereby, NeuMF gets even better performance. Interestingly, in both datasets, LightGCN's results are not remarkable, and we believe that without using the attention mechanism, the discrimination for many neighboring nodes becomes worse.

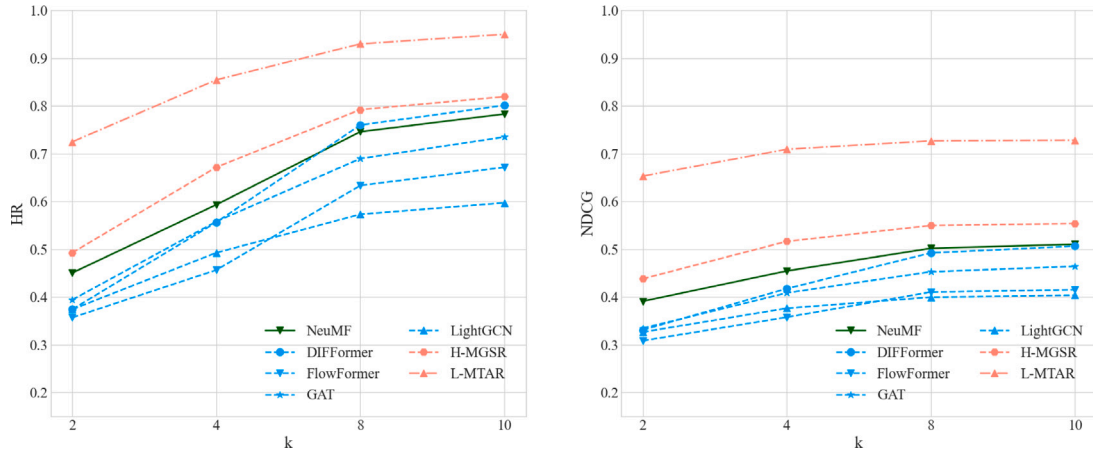


Fig. 9. HR and NDCG results in MovieLens dataset.

In comparison to all the baselines, our model demonstrates superior performance and lower variance across all evaluation metrics. We attribute these enhancements to four crucial factors in MTAR's design: Firstly, the incorporation of higher-order relationships in PE aids the model in extracting and integrating hidden relations between nodes more effectively. Secondly, the excellent semantic extraction of fine-tuned LLM. Thirdly, the utilization of neighboring attention captures essential local information crucial for the recommendation task. Lastly, the graph Transformer proves highly effective in addressing the limitations of neighbor relationships inherent in MPNNs by fusing information across the entire graph scope.

### 5.5. Ablation study

We conduct ablation experiments to assess the impact of each component of the model. Specifically, we perform six ablation experiments:

1. Without using the motifs, consider only direct neighbors when encoding locations. The dimension of the  $F'$  remains at 1024, consistent with the configuration when motifs are utilized.
2. Without local information, relying on global information in the PE process. This implies that the attention mechanism between neighboring nodes is not employed.
3. Without using semantic encoding, only positional encoding is used.
4. Without local information and semantic encoding, only motif-based global positional encoding is used.
5. Without using linearizing attention, based on (2), we replaced the attention module with a Vanilla Transformer (abbreviated as Vanilla in the table) to test the efficiency improvement. For extensive evaluation, we additionally randomly generated two data with 2K and 4K nodes respectively.
6. Since there is a large variety of combinations of motifs, we randomized different combinations of motifs to verify the robustness of the model.

In each experiment, the remaining settings are kept unchanged from the full model, with only one component removed. The results are presented in Fig. 10, Table 5, and 6. Across both datasets, our experiments demonstrate a consistent trend. Firstly, considering local and higher-order relations in PE enhances our model's understanding of potential information between nodes. Secondly, experiment 2 shows more decay compared to experiment 1, indicating the significance of local connections between nodes in recommendations. Disregarding local relationships in PE is not feasible, and the use of motifs proves effective in capturing higher-order information between nodes. Thirdly, employing semantic coding can enhance the model's performance. Meanwhile,

Table 5

Ablation study on two datasets.

| ProgrammableWeb | HR@4          | HR@10         | NDCG@4        | NDCG@10       |
|-----------------|---------------|---------------|---------------|---------------|
| Full model      | <b>0.9262</b> | <b>0.9306</b> | <b>0.9137</b> | <b>0.9081</b> |
| Experiment (1)  | 0.8691        | 0.9025        | 0.8199        | 0.8254        |
| Experiment (2)  | 0.5685        | 0.7065        | 0.4765        | 0.5225        |
| Experiment (3)  | 0.9236        | 0.9279        | 0.9090        | 0.9032        |
| Experiment (4)  | 0.5483        | 0.6951        | 0.4641        | 0.5133        |
| MovieLens       | HR@4          | HR@10         | NDCG@4        | NDCG@10       |
| Full model      | <b>0.8543</b> | <b>0.9497</b> | <b>0.7090</b> | <b>0.7282</b> |
| Experiment (1)  | 0.8191        | 0.9367        | 0.6630        | 0.6912        |
| Experiment (2)  | 0.6422        | 0.8020        | 0.5090        | 0.5480        |
| Experiment (3)  | 0.8312        | 0.9477        | 0.6822        | 0.7047        |
| Experiment (4)  | 0.6281        | 0.8020        | 0.5037        | 0.5464        |

Table 6

Comparison of time consumption.

| Module  | Train speed (seconds per epoch) |        |                 |           |
|---------|---------------------------------|--------|-----------------|-----------|
|         | 2K                              | 4K     | ProgrammableWeb | MovieLens |
| Vanilla | 0.0236                          | 0.0502 | 0.1073          | 0.1478    |
| Ours    | 0.0159                          | 0.0238 | 0.0349          | 0.0451    |

Experiment 4 demonstrates that although the use of higher-order relations in the complete method can effectively improve accuracy, it is not sufficient on its own without an effective method for information aggregation. In experiment 5, we counted the time consumption of 500 epochs and showed the data in Table 6. The higher the number of nodes, the more time our model saves. Finally, in experiment 6, the results are illustrated and compared in Fig. 10. It turns out that, although the results of the L-MTAR show some differences for different combinations of motifs, the overall results are still ahead of the other approaches. This proves that the choice of motifs does not significantly affect the L-MTAR.

Also, we find that the model performs slightly differently across the two datasets when motifs are not used, performing significantly better on the ProgrammableWeb dataset than on the MovieLens dataset. Therefore, we briefly analyze the average number of different types of neighbors per node in both datasets, as shown in Fig. 11. In the MovieLens dataset, we believe the direct neighbor relationships between nodes are relatively dense, potentially containing more redundancy or errors, which may lead to this result. As a result, the use of motifs and local relations can be beneficial in both complex and sparse scenarios.

## 6. Conclusion

Fast and accurate API recommendations are crucial to address the rapid growth of the number of Web services and the diverse needs of

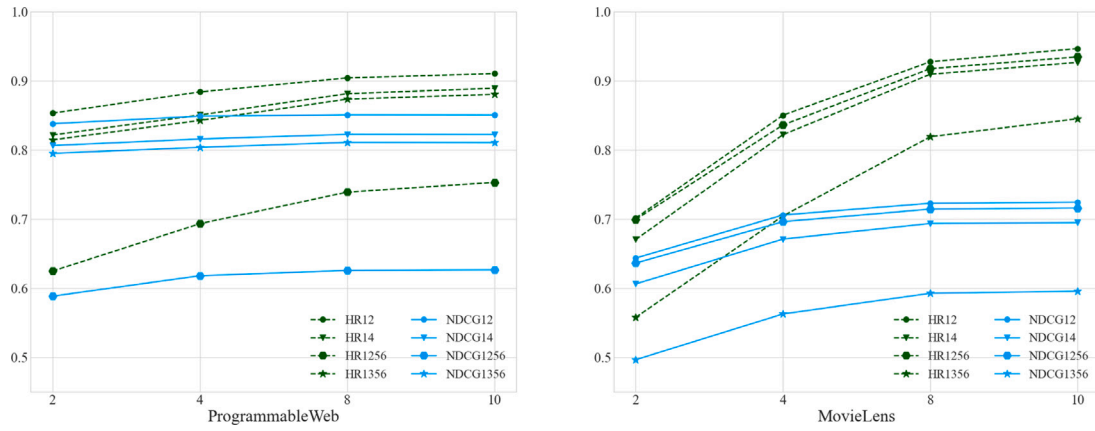


Fig. 10. Experiments are conducted in both datasets using different combinations of motifs, and the numbers following the metrics represent the type of motif we used. e.g., HR12 represents the HR results when using  $M_1$  and  $M_2$ .

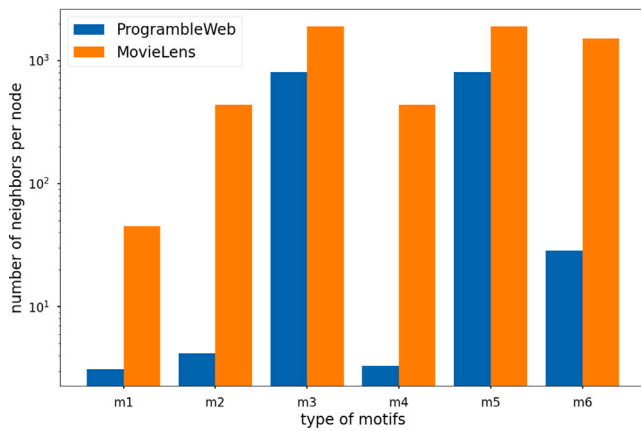


Fig. 11. An illustration of the average number of different types of neighbors that each node has in both datasets.

people today. This paper introduces a linearizing graph Transformer named L-MTAR for Web API recommendation. It incorporates a PE method that exploits motifs and text information. Unlike some existing models that overlook the higher-order structures or text data within graphs, L-MTAR effectively harnesses fine-tuned LLM and hierarchical relations in the PE process. Meanwhile, we designed a linear time complexity graph Transformer using the higher-order information between nodes. Experimental results on real datasets demonstrate that L-MTAR outperforms state-of-the-art models. Additionally, ablation experiments are conducted to analyze the effectiveness of the gains from higher-order relations. It also demonstrates that our model is more applicable to the recommendation of Web APIs and can improve efficiency. Looking ahead, we have the potential to combine other models to extract unutilized information between nodes.

#### CRedit authorship contribution statement

**Xin Zheng:** Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Guiling Wang:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Funding acquisition, Conceptualization. **Guiyue Xu:** Software, Methodology. **Jianye Yang:** Validation, Software, Investigation. **Boyang Han:** Writing – review & editing, Validation. **Jian Yu:** Supervision, Methodology, Formal analysis, Data curation.

#### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

#### Acknowledgments

This work is supported by Projects of International Cooperation and Exchanges NSFC (Grant No. 62061136006).

#### Data availability

Data will be made available on request.

#### References

- [1] J.S. Breese, D. Heckerman, C. Kadie, Empirical analysis of predictive algorithms for collaborative filtering, in: Proceedings of the Fourteenth Conference on Uncertainty in Artificial Intelligence, UAI '98, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1998, pp. 43–52.
- [2] J. Bobadilla, F. Ortega, A. Hernando, A. Gutiérrez, Recommender systems survey, Knowl.-Based Syst. 46 (2013) 109–132, <http://dx.doi.org/10.1016/j.knsys.2013.03.012>.
- [3] C. Gao, Y. Zheng, N. Li, Y. Li, Y. Qin, J. Piao, Y. Quan, J. Chang, D. Jin, X. He, Y. Li, A survey of graph neural networks for recommender systems: Challenges, methods, and directions, ACM Trans. Recomm. Syst. 1 (1) (2023) <http://dx.doi.org/10.1145/3568022>.
- [4] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, 2017, [arXiv:1609.02907](https://arxiv.org/abs/1609.02907).
- [5] M. Nguyen, J. Yu, T. Nguyen, Y. Han, Attentional matrix factorization with context and co-invocation for service recommendation, Expert Syst. Appl. 186 (2021) 115698, <http://dx.doi.org/10.1016/j.eswa.2021.115698>.
- [6] Y. Ma, X. Geng, J. Wang, A deep neural network with multiplex interactions for cold-start service recommendation, IEEE Trans. Eng. Manage. 68 (1) (2021) 105–119, <http://dx.doi.org/10.1109/TEM.2019.2961376>.
- [7] M. Nguyen, J. Yu, T. Nguyen, S. Yongchareon, High-order autoencoder with data augmentation for collaborative filtering, Knowl.-Based Syst. 240 (2022) 107773, <http://dx.doi.org/10.1016/j.knsys.2021.107773>.
- [8] G. Wang, J. Yu, M. Nguyen, Y. Zhang, S. Yongchareon, Y. Han, Motif-based graph attentional neural network for web service recommendation, Knowl.-Based Syst. 269 (2023) 110512, <http://dx.doi.org/10.1016/j.knsys.2023.110512>.
- [9] X. Zheng, G. Wang, J. Zhang, Y. Zhang, N. Wang, J. Yu, Y. Han, H-MGSR: A hierarchical motif-based graph attention neural network for service recommendation, in: 2023 IEEE International Conference on Web Services, ICWS, 2023, pp. 553–562, <http://dx.doi.org/10.1109/ICWS60048.2023.00074>.
- [10] K. Duan, Q. Liu, T.-S. Chua, S. Yan, W.T. Ooi, Q. Xie, J. He, SimTeG: A frustratingly simple approach improves textual graph learning, 2023, [arXiv:2308.02565](https://arxiv.org/abs/2308.02565).
- [11] Y. Du, Z. Wang, Z. Sun, H. Chua, H. Liu, Z. Wu, Y. Ma, J. Zhang, Y. Sun, Large language model with graph convolution for recommendation, 2024, [arXiv:2402.08859](https://arxiv.org/abs/2402.08859).

- [12] H. Pei, B. Wei, K.C.-C. Chang, Y. Lei, B. Yang, Geom-GCN: Geometric graph convolutional networks, 2020, [arXiv:2002.05287](#).
- [13] D. Kreuzer, D. Beaini, W.L. Hamilton, V. Létourneau, P. Tossou, Rethinking graph transformers with spectral attention, 2021, [arXiv:2106.03893](#).
- [14] Q. Wu, W. Zhao, Z. Li, D. Wipf, J. Yan, NodeFormer: A scalable graph structure learning transformer for node classification, in: *Advances in Neural Information Processing Systems, NeurIPS*, 2022.
- [15] H. Wu, J. Wu, J. Xu, J. Wang, M. Long, Flowformer: Linearizing transformers with conservation flows, in: *International Conference on Machine Learning*, 2022.
- [16] V.P. Dwivedi, A.T. Luu, T. Laurent, Y. Bengio, X. Bresson, Graph neural networks with learnable structural and positional representations, 2022, [arXiv:2110.07875](#).
- [17] J. Zhang, H. Zhang, C. Xia, L. Sun, Graph-Bert: Only attention is needed for learning graph representations, 2020, *CoRR* [abs/2001.05140](#). [arXiv:2001.05140](#).
- [18] V.P. Dwivedi, X. Bresson, A generalization of transformer networks to graphs, 2020, *CoRR* [abs/2012.09699](#). [arXiv:2012.09699](#).
- [19] L. Wu, Z. Qiu, Z. Zheng, H. Zhu, E. Chen, Exploring large language model for graph data understanding in online job recommendations, 2023, [arXiv:2307.05722](#).
- [20] Y. Feng, H. You, Z. Zhang, R. Ji, Y. Gao, Hypergraph neural networks, in: *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence*, in: *AAAI'19/IAAI'19/EAAI'19*, AAAI Press, 2019, [http://dx.doi.org/10.1609/aaai.v33i01.33013558](#).
- [21] J. Tang, Y. Yang, W. Wei, L. Shi, L. Su, S. Cheng, D. Yin, C. Huang, GraphGPT: Graph instruction tuning for large language models, 2024, [arXiv:2310.13023](#).
- [22] Z. Guo, L. Xia, Y. Yu, Y. Wang, Z. Yang, W. Wei, L. Pang, T.-S. Chua, C. Huang, GraphEdit: Large language models for graph structure learning, 2024, [arXiv:2402.15183](#).
- [23] R. Milo, S. Shen-Orr, S. Itzkovitz, N. Kashtan, D. Chklovskii, U. Alon, Network motifs: Simple building blocks of complex networks, *Science* 298 (5594) (2002) 824–827, [http://dx.doi.org/10.1126/science.298.5594.824](#), [arXiv:https://www.science.org/doi/pdf/10.1126/science.298.5594.824](#).
- [24] Y. Zhang, H. Yang, L. Kuang, A web API recommendation method with composition relationship based on GCN, 2020, pp. 601–608, [http://dx.doi.org/10.1109/ISPA-BDCloud-SocialCom-SustainCom51426.2020.00101](#).
- [25] J. Gilmer, S.S. Schoenholz, P.F. Riley, O. Vinyals, G.E. Dahl, Neural message passing for quantum chemistry, in: D. Precup, Y.W. Teh (Eds.), *Proceedings of the 34th International Conference on Machine Learning*, in: *Proceedings of Machine Learning Research*, vol. 70, PMLR, 2017, pp. 1263–1272.
- [26] J. Zhou, G. Cui, Z. Zhang, C. Yang, Z. Liu, M. Sun, Graph neural networks: A review of methods and applications, 2018, *CoRR* [abs/1812.08434](#). [arXiv:1812.08434](#).
- [27] X. Liu, L. Huang, V. Ng, Effective API recommendation without historical software repositories, in: *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, ASE '18*, Association for Computing Machinery, New York, NY, USA, 2018, pp. 282–292, [http://dx.doi.org/10.1145/3238147.3238216](#).
- [28] J. Sun, Y. Zhang, W. Guo, H. Guo, R. Tang, X. He, C. Ma, M. Coates, Neighbor interaction aware graph convolution networks for recommendation, in: *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '20*, Association for Computing Machinery, New York, NY, USA, 2020, pp. 1289–1298, [http://dx.doi.org/10.1145/3397271.3401123](#).
- [29] S. Wang, L. Hu, Y. Wang, X. He, Q.Z. Sheng, M.A. Orgun, L. Cao, F. Ricci, P.S. Yu, Graph learning based recommender systems: A review, 2021, *CoRR* [abs/2105.06339](#). [arXiv:2105.06339](#). URL [https://arxiv.org/abs/2105.06339](#).
- [30] Y. Yu, Z. Lu, J. Liu, G. Zhao, J.-r. Wen, RUM: Network representation learning using motifs, in: *2019 IEEE 35th International Conference on Data Engineering, ICDE*, 2019, pp. 1382–1393, [http://dx.doi.org/10.1109/ICDE.2019.00125](#).
- [31] T. Wang, J. Peng, Q. Peng, Y. Wang, J. Chen, FSM: Fast and scalable network motif discovery for exploring higher-order network organizations, *Methods* 173 (2020) 83–93, [http://dx.doi.org/10.1016/j.ymeth.2019.07.008](#), Multiscale Network-based Approaches.
- [32] T.B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D.M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, 2020, [arXiv:2005.14165](#).
- [33] Z. Du, Y. Qian, X. Liu, M. Ding, J. Qiu, Z. Yang, J. Tang, GLM: General language model pretraining with autoregressive blank infilling, 2022, [arXiv:2103.10360](#).
- [34] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, E.H. Chi, T. Hashimoto, O. Vinyals, P. Liang, J. Dean, W. Fedus, Emergent abilities of large language models, 2022, [http://dx.doi.org/10.48550/ARXIV.2206.07682](#), *CoRR* [abs/2206.07682](#). [arXiv:2206.07682](#). URL [https://doi.org/10.48550/ARXIV.2206.07682](#).
- [35] B. Jin, G. Liu, C. Han, M. Jiang, H. Ji, J. Han, Large language models on graphs: A comprehensive survey, 2024, [arXiv:2312.02783](#).
- [36] H. Wang, S. Feng, T. He, Z. Tan, X. Han, Y. Tsvetkov, Can language models solve graph problems in natural language? 2024, [arXiv:2305.10037](#).
- [37] C. Liu, B. Wu, Evaluating large language models on graphs: Performance insights and comparative analysis, 2023, [arXiv:2308.11224](#).
- [38] J. Zhu, Y. Cui, Y. Liu, H. Sun, X. Li, M. Pelger, T. Yang, L. Zhang, R. Zhang, H. Zhao, TextGNN: Improving text encoder via graph neural network in sponsored search, in: *Proceedings of the Web Conference 2021, WWW '21*, ACM, 2021, [http://dx.doi.org/10.1145/3442381.3449842](#).
- [39] B. Fatemi, J. Halcrow, B. Perozzi, Talk like a graph: Encoding graphs for large language models, 2023, [arXiv:2310.04560](#).
- [40] G. Mialon, D. Chen, M. Selosse, J. Mairal, GraphiT: Encoding graph structure in transformers, 2021, *CoRR* [abs/2106.05667](#). [arXiv:2106.05667](#).
- [41] D. Chen, L. O'Bray, K. Borgwardt, Structure-aware transformer for graph representation learning, 2022, [arXiv:2202.03036](#).
- [42] Q. Wu, C. Yang, W. Zhao, Y. He, D. Wipf, J. Yan, DIFFormer: Scalable (graph) transformers induced by energy constrained diffusion, in: *International Conference on Learning Representations, ICLR*, 2023.
- [43] Z. Qin, W. Sun, H. Deng, D. Li, Y. Wei, B. Lv, J. Yan, L. Kong, Y. Zhong, cosFormer: Rethinking softmax in attention, in: *International Conference on Learning Representations*, 2022.
- [44] B. Zheng, Y. Hou, H. Lu, Y. Chen, W.X. Zhao, M. Chen, J.-R. Wen, Adapting large language models by integrating collaborative semantics for recommendation, 2024, [arXiv:2311.09049](#).
- [45] Z. Du, Y. Qian, X. Liu, M. Ding, J. Qiu, Z. Yang, J. Tang, All NLP tasks are generation tasks: A general pretraining framework, 2021, *CoRR* [abs/2103.10360](#). [arXiv:2103.10360](#). URL [https://arxiv.org/abs/2103.10360](#).
- [46] E.J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, W. Chen, LoRA: Low-rank adaptation of large language models, 2021, *CoRR* [abs/2106.09685](#). [arXiv:2106.09685](#). URL [https://arxiv.org/abs/2106.09685](#).
- [47] L. McInnes, J. Healy, J. Melville, UMAP: Uniform manifold approximation and projection for dimension reduction, 2020, [arXiv:1802.03426](#).
- [48] L. Rampásek, M. Galkin, V.P. Dwivedi, A.T. Luu, G. Wolf, D. Beaini, Recipe for a general, powerful, scalable graph transformer, *Adv. Neural Inf. Process. Syst.* 35 (2022).
- [49] V.P. Dwivedi, C.K. Joshi, T. Laurent, Y. Bengio, X. Bresson, Benchmarking graph neural networks, 2020, *CoRR* [abs/2003.00982](#). [arXiv:2003.00982](#).
- [50] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, 2018, [arXiv:1710.10903](#).
- [51] F.M. Harper, J.A. Konstan, The MovieLens datasets: History and context, *ACM Trans. Interact. Intell. Syst.* 5 (4) (2015) [http://dx.doi.org/10.1145/2827872](#).
- [52] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, M. Wang, LightGCN: Simplifying and powering graph convolution network for recommendation, 2020, *CoRR* [abs/2002.02126](#). [arXiv:2002.02126](#).
- [53] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, T. Chua, Neural collaborative filtering, 2017, *CoRR* [abs/1708.05031](#). [arXiv:1708.05031](#). URL [http://arxiv.org/abs/1708.05031](#).